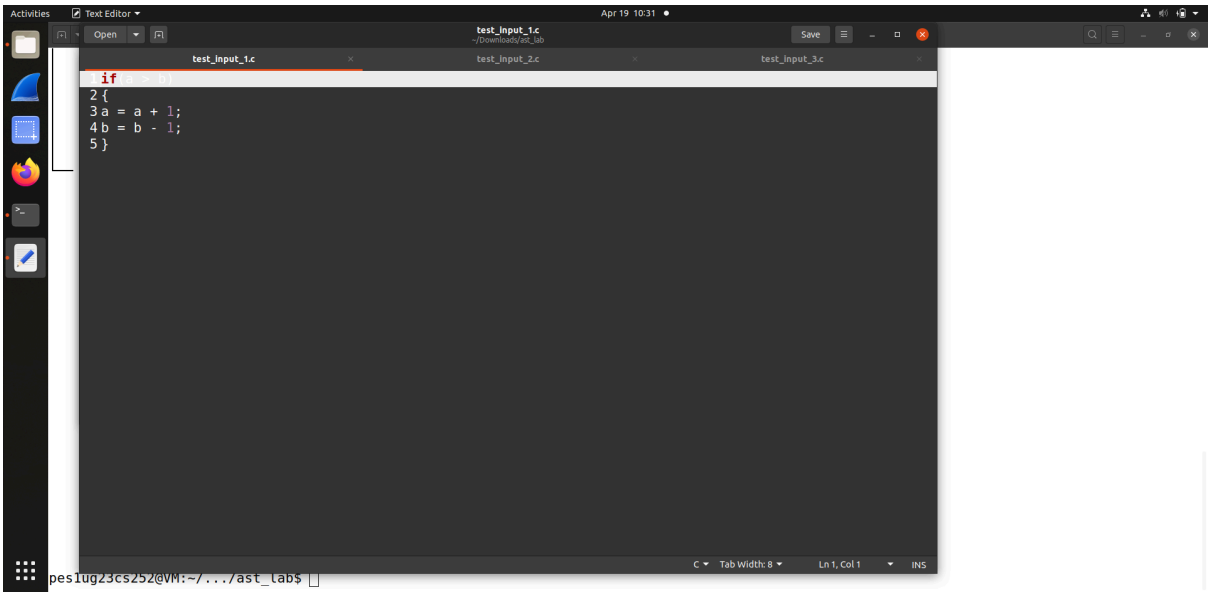
CODING ASSIGNMENT 2

ISL NANDINI
PES1UG23CS252
SECTION E

AST

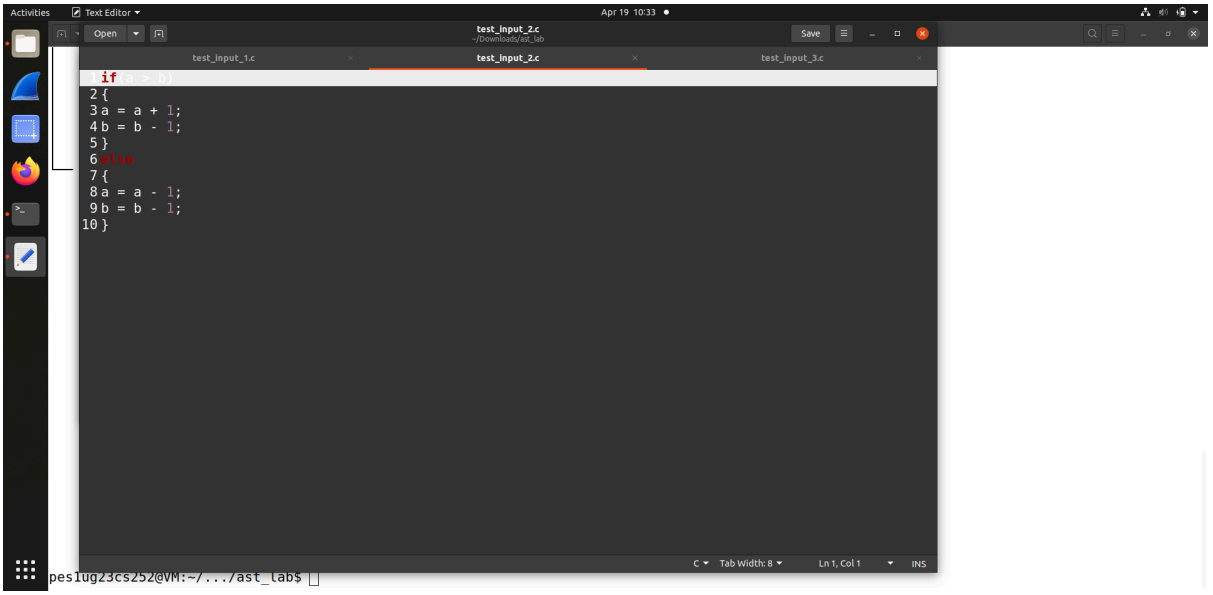
SS OF INPUT 1



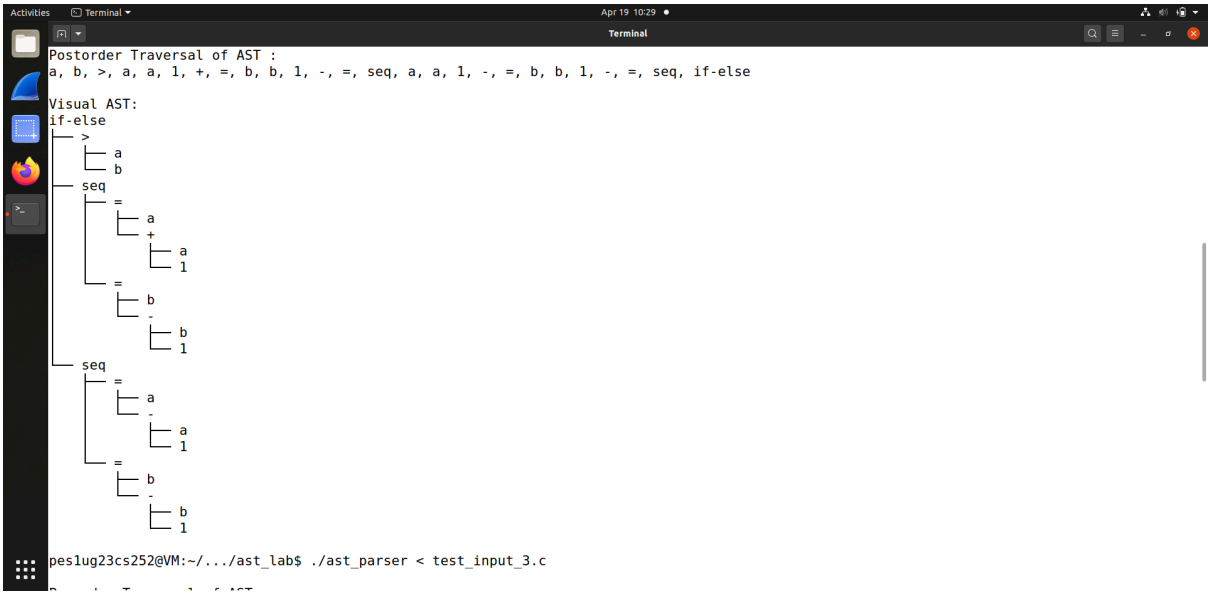
```
 inflating: ast_lab/test_input_3.c
pes1ug23cs252@VM:~/Downloads$ cd ast_lab
pes1ug23cs252@VM:~/.../ast_lab$ flex -o lex.yy.c lexer.l
pes1ug23cs252@VM:~/.../ast_lab$ yacc -d parser.y
pes1ug23cs252@VM:~/.../ast_lab$ gcc -g y.tab.c lex.yy.c -o ast_parser
pes1ug23cs252@VM:~/.../ast_lab$ ./ast_parser < test_input_1.c

Preorder Traversal of AST :
if, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1

Postorder Traversal of AST :
a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, if

Visual AST:
if
├── >
│   ├── a
│   └── b
└── seq
    ├── =
    │   ├── a
    │   └── +
    │       ├── a
    │       └── 1
    └── =
        ├── b
        └── -
            ├── b
            └── 1
```

Simple `if` statement The input contains a simple `if` statement that checks whether `a > b`. If the condition is true, two assignment statements are executed: `a = a + 1` and `b = b - 1`. The AST is built with `if` as the root node, having the condition > as its left child and a `seq` node as its right child. The `seq` node represents the sequential execution of the two assignment statements, each of which is an = node with the variable on the left and the arithmetic expression on the right. The preorder traversal visits the root before its children, giving: `if, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1`. The postorder traversal visits children before the root, giving: `a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, if`.

SS OF INPUT 2

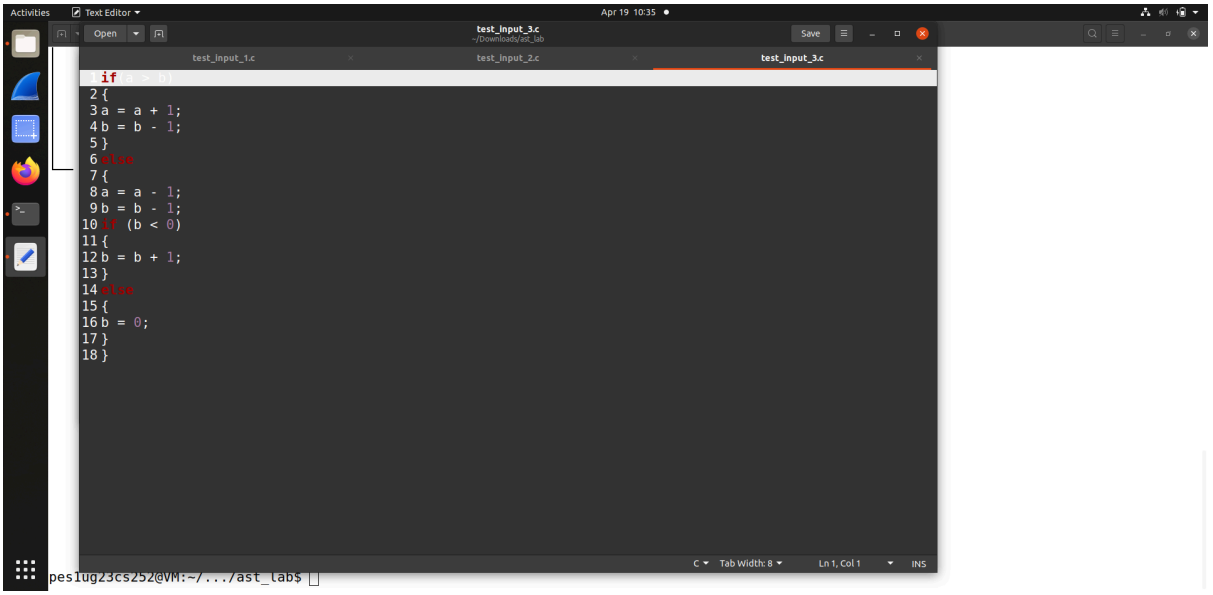


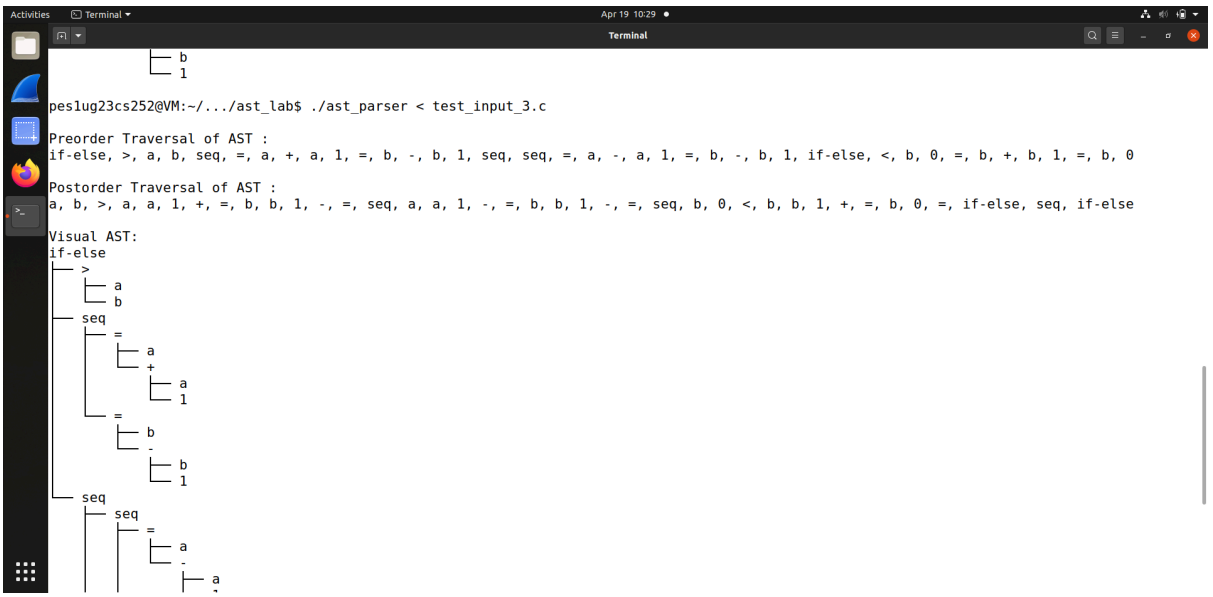

```
 inflating: ast_lab/test_input_3.c
pes1ug23cs252@VM:~/Downloads$ cd ast_lab
pes1ug23cs252@VM:~/.../ast_lab$ flex -o lex.yy.c lexer.l
pes1ug23cs252@VM:~/.../ast_lab$ yacc -d parser.y
pes1ug23cs252@VM:~/.../ast_lab$ gcc -g y.tab.c lex.yy.c -o ast_parser
pes1ug23cs252@VM:~/.../ast_lab$ ./ast_parser < test_input_1.c

Preorder Traversal of AST :
if, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1

Postorder Traversal of AST :
a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, if

Visual AST:
if
├── >
│   ├── a
│   └── b
└── seq
    ├── =
    │   ├── a
    │   └── +
    │       ├── a
    │       └── 1
    └── =
        ├── b
        └── -
            ├── b
            └── 1
```

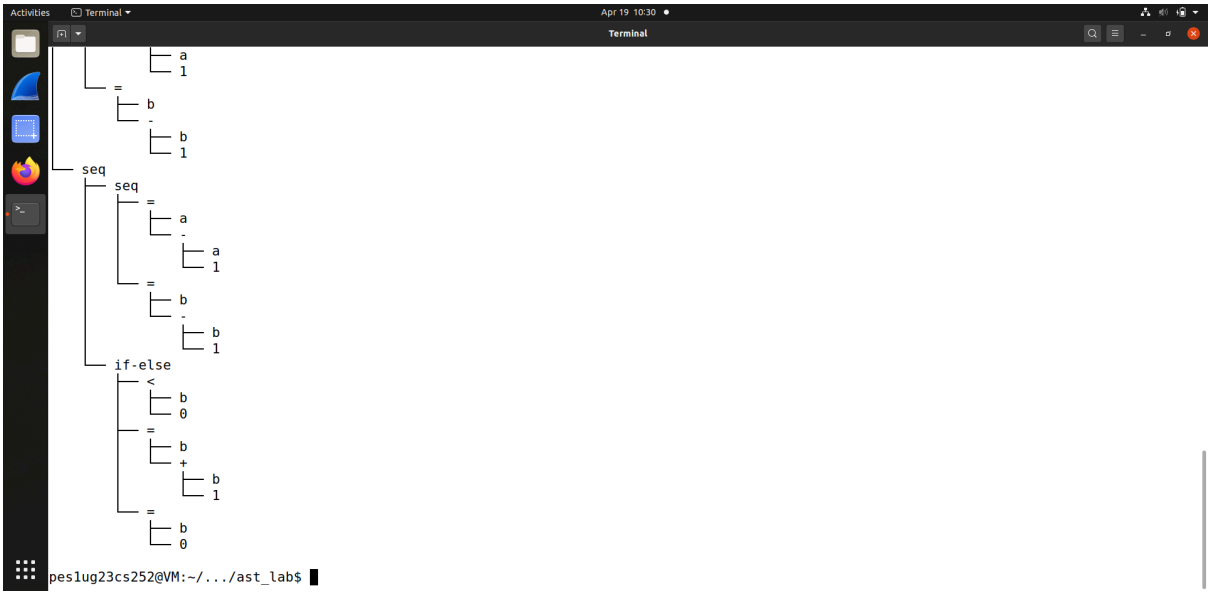


Input 2 – `if-else` statement The input extends Input 1 by adding an `else` block containing `a = a - 1` and `b = b - 1`. The AST root becomes `if-else`, a three-child node holding the condition >, the then-block `seq`, and the else-block `seq`. Each `seq` node chains the two assignment statements within that branch. The preorder traversal gives: `if-else, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1, seq, =, a, -, a, 1, =, b, -, b, 1`. The postorder traversal gives: `a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, a, a, 1, -, =, b, b, 1, -, =, seq, if-else`.

SS3


Activities
Text Editor
Apr 19 10:35
Open
test_input_3.c
~/Downloads/ss_lab
Save
test_input_1.c
test_input_2.c
test_input_3.c
if (a > b)
{
a = a + 1;
b = b - 1;
}
else
{
a = a - 1;
b = b - 1;
if (b < 0)
{
b = b + 1;
}
else
{
b = 0;
}
}
C
Tab Width: 8
Ln 1, Col 1
INS
pes1ug23cs252@VM:~/.../ast_lab$

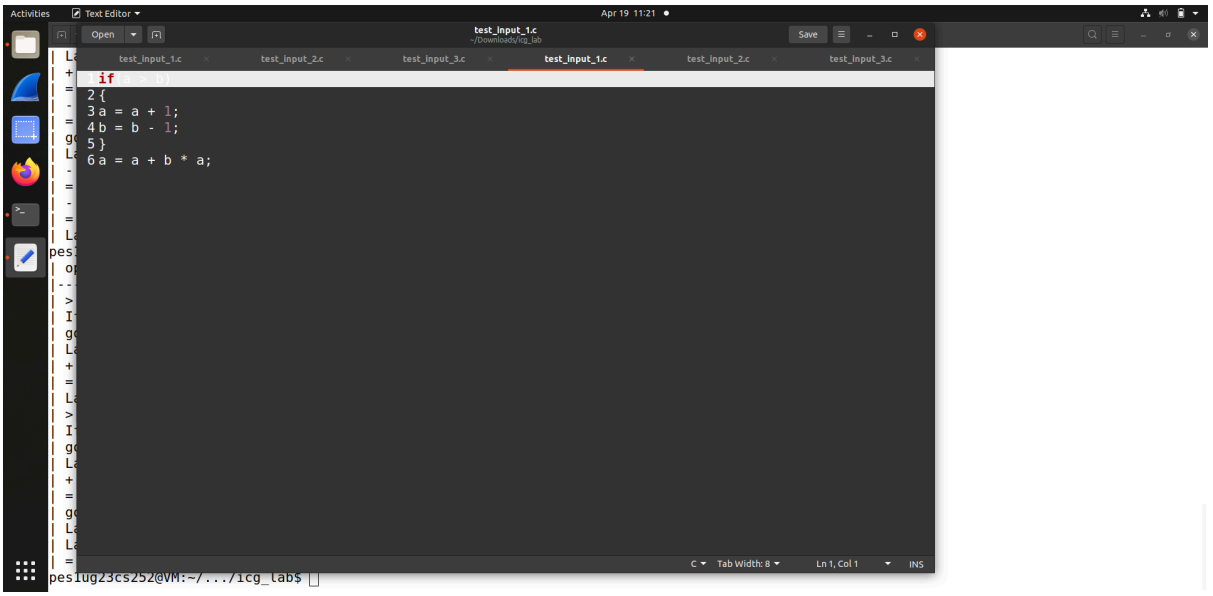

Activities
Terminal
Apr 19 10:29
Terminal
├── b
└── 1
pes1ug23cs252@VM:~/.../ast_lab$ ./ast_parser < test_input_3.c
Preorder Traversal of AST :
if-else, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1, seq, seq, =, a, -, a, 1, =, b, -, b, 1, if-else, <, b, 0, =, b, +, b, 1, =, b, 0
Postorder Traversal of AST :
a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, a, a, 1, -, =, b, b, 1, -, =, seq, b, 0, <, b, b, 1, +, =, b, 0, =, if-else, seq, if-else
Visual AST:
if-else
├── >
│ ├── a
│ └── b
├── seq
│ ├── =
│ │ ├── a
│ │ └── +
│ │ ├── a
│ │ └── 1
│ └── =
│ ├── b
│ └── -
│ ├── b
│ └── 1
└── seq
├── seq
│ ├── =
│ │ ├── a
│ │ └── -
│ │ ├── a



Nested `if-else` The input introduces a nested `if-else` inside the outer `else` block. The outer `else` block now contains three statements: `a = a - 1`, `b = b - 1`, and an inner `if-else` that checks `b < 0` — if true it executes `b = b + 1`, otherwise `b = 0`. The outer else-block is represented as a `seq` node chaining the first two assignments with the inner `if-else` node. This demonstrates how the AST naturally captures nesting through recursive tree structure. The preorder traversal gives: `if-else, >, a, b, seq, =, a, +, a, 1, =, b, -, b, 1, seq, seq, =, a, -, a, 1, =, b, -, b, 1, if-else, <, b, 0, =, b, +, b, 1, =, b, 0`. The postorder traversal gives: `a, b, >, a, a, 1, +, =, b, b, 1, -, =, seq, a, a, 1, -, =, b, b, 1, -, =, seq, b, 0, <, b, b, 1, +, =, b, 0, =, if-else, seq, if-else`.

ICG

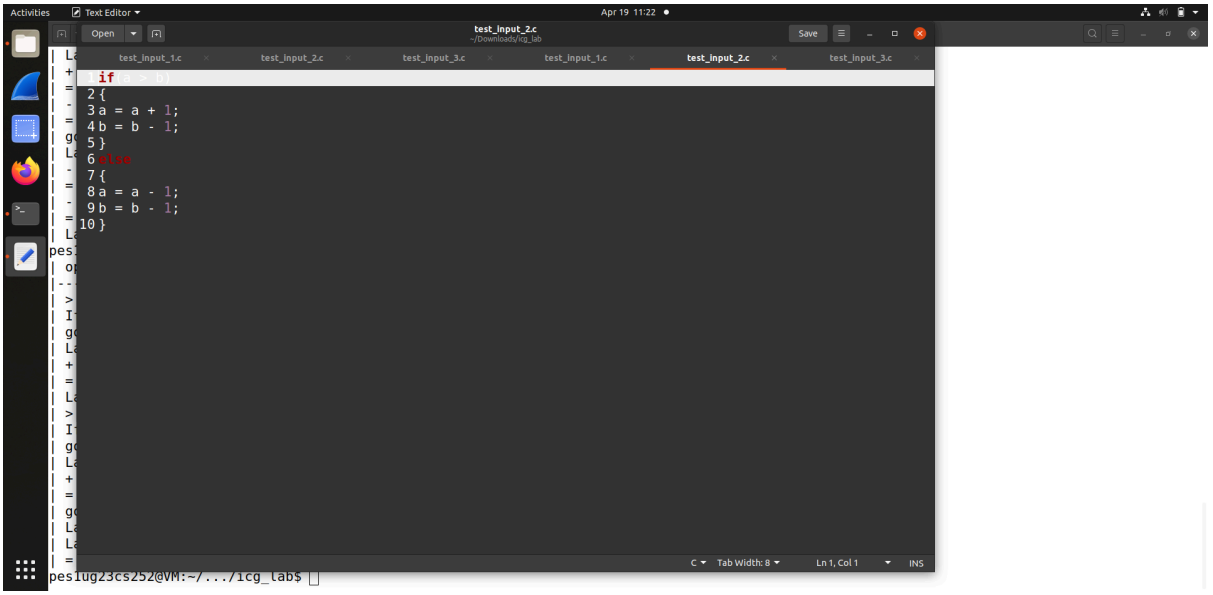
SS1




```
pes1ug23cs252@VM:~/.../icg_lab$ ./icg_parser < test_input_1.c
| op      | arg1    | arg2    | result    |
|---------|---------|---------|-----------|
| >       | a       | b       | t1        |
| If      | t1      |         | L1        |
| goto    |         |         | L2        |
| Label   |         |         | L1        |
| +       | a       | 1       | t2        |
| =       | t2      |         | a         |
| -       | b       | 1       | t3        |
| =       | t3      |         | b         |
| Label   |         |         | L2        |
| *       | b       | a       | t4        |
| +       | a       | t4      | t5        |
| =       | t5      |         | a         |
```

`if` statement followed by an assignment The input contains an `if` statement that checks whether a `>` b, and if true executes a `=` a `+` 1 and b `=` b `-` 1. After the `if` block, there is a standalone assignment a `=` a `+` b `*` a. The three-address code generation begins by evaluating the condition a `>` b and storing the result in `t1`. A conditional jump `if t1 goto` L1 is emitted, meaning if the condition is true execution jumps to label L1 (the then-block). If false, a `goto` L2 skips the block entirely. Inside L1, temporaries `t2` and `t3` are used to compute the assignments. After the if-block, L2 marks the start of the post-if code, where the expression b `*` a is computed into `t4`, then a `+` `t4` into `t5`, and finally assigned back to a.

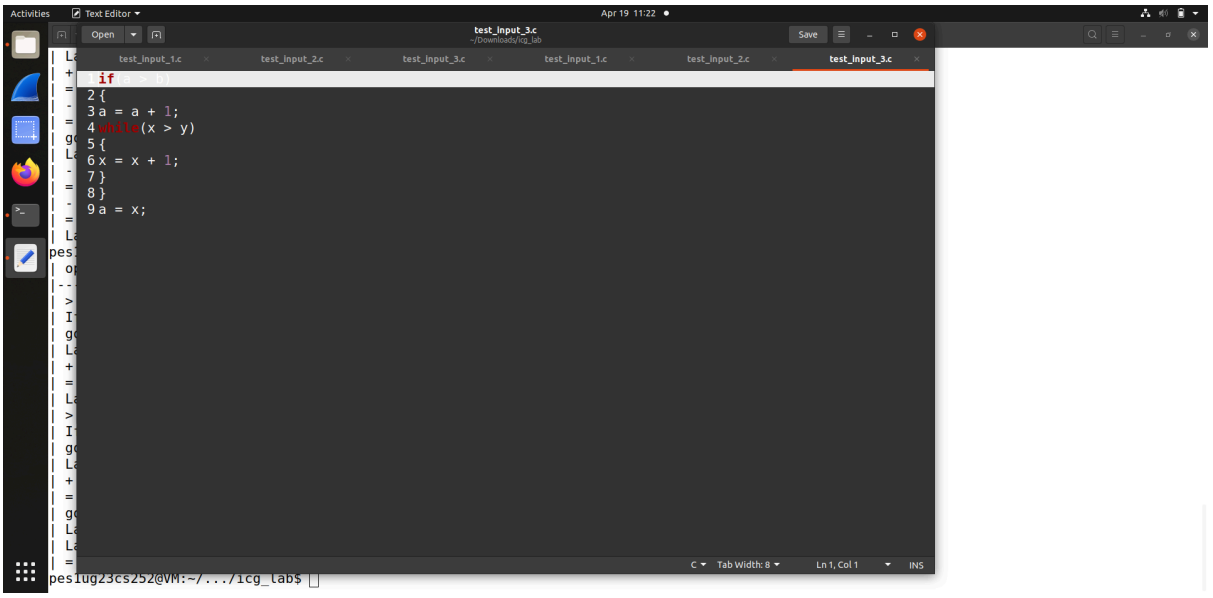
SS2




```
pes1ug23cs252@VM:~/.../icg_lab$ ./icg_parser < test_input_2.c
| op      | arg1    | arg2    | result    |
|---------|---------|---------|-----------|
| >       | a       | b       | t1        |
| If      | t1      |         | L1        |
| goto    |         |         | L2        |
| Label   |         |         | L1        |
| +       | a       | 1       | t2        |
| =       | t2      |         | a         |
| -       | b       | 1       | t3        |
| =       | t3      |         | b         |
| goto    |         |         | L3        |
| Label   |         |         | L2        |
| -       | a       | 1       | t4        |
| =       | t4      |         | a         |
| -       | b       | 1       | t5        |
| =       | t5      |         | b         |
| Label   |         |         | L3        |
```

`if-else` statement The input has an `if-else` where the then-branch executes `a = a + 1` and `b = b - 1`, and the else-branch executes `a = a - 1` and `b = b - 1`. The condition `a > b` is evaluated into `t1`. A conditional jump sends execution to L1 (then-block) if true, otherwise falls through to `goto L2` (else-block). After the then-block completes, a `goto L3` is emitted to jump past the else-block, which is critical to prevent the else-block from executing after the then-block. L2 marks the start of the else-block where temporaries `t4` and `t5` handle the computations. Finally L3 marks the end of the entire if-else construct, and both branches converge here.

SS3




```
pes1ug23cs252@VM:~/.../icg_lab$ ./icg_parser < test_input_3.c
| op      | arg1    | arg2    | result    |
|---------|---------|---------|-----------|
| >       | a       | b       | t1        |
| If      | t1      |         | L1        |
| goto    |         |         | L2        |
| Label   |         |         | L1        |
| +       | a       | 1       | t2        |
| =       | t2      |         | a         |
| Label   |         |         | L3        |
| >       | x       | y       | t3        |
| If      | t3      |         | L4        |
| goto    |         |         | L5        |
| Label   |         |         | L4        |
| +       | x       | 1       | t4        |
| =       | t4      |         | x         |
| goto    |         |         | L3        |
| Label   |         |         | L5        |
| Label   |         |         | L2        |
| =       | x       |         | a         |
pes1ug23cs252@VM:~/.../icg_lab$
```

`if` statement with a nested `while` loop The input has an `if` block containing an assignment `a = a + 1` followed by a `while(x > y)` loop that increments `x`. After the entire `if` block, `a = x` is assigned. The outer `if` generates `t1` for `a > b` and jumps to L1 if true, else `goto` L2. Inside the then-block, after computing `a = a + 1`, the while loop begins at label L3 which is the loop re-entry point. The condition `x > y` is evaluated into `t3` — if true, execution jumps to L4 (loop body), otherwise `goto L5` exits the loop. Inside L4, `x = x + 1` is computed via `t4`, and a `goto L3` sends execution back to re-evaluate the condition. L5 marks the while loop exit, and L2 marks the end of the outer if-block. Finally `a = x` is emitted as a simple copy assignment.